

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

In the futuristic city of **Energon Prime**, engineers have discovered a powerful **P-digit** Energy Code(Number) that controls the entire city's power grid. This code must be carefully analyzed to unlock its hidden energy potential. The system responds only to a **special digit N** embedded within the code.

As you examine the Energy Code from Right to Left:

- If the digit N is not found anywhere in the number, then the system shuts down and no energy is generated.
- If the digit N is found, the system activates. To compute the energy, the system uses a layered frequency analysis method. For each digit from 0 to 9, the system re-scans the entire number to count its occurrences and print frequency. The energy contribution of each digit is calculated as $\text{digit} \times (\text{frequency})$. All nonzero contributions are multiplied to get the **final energy**.

Write a Java program to print whether N was found and the resulting energy value, or indicate that no energy is generated.

Sample Input 1	Sample Output 1
122331 3	Digit 0 -> Frequency: 0, Contribution: 0 Digit 1 -> Frequency: 2, Contribution: 2 Digit 2 -> Frequency: 2, Contribution: 4 Digit 3 -> Frequency: 2, Contribution: 6 Digit 4 -> Frequency: 0, Contribution: 0 Digit 5 -> Frequency: 0, Contribution: 0 Digit 6 -> Frequency: 0, Contribution: 0 Digit 7 -> Frequency: 0, Contribution: 0 Digit 8 -> Frequency: 0, Contribution: 0 Digit 9 -> Frequency: 0, Contribution: 0 Total Energy: 48 Explanation: Frequency count: 1 → 2 times, 2 → 2 times, 3 → 2 times $(1 \times 2) \times (2 \times 2) \times (3 \times 2)$ = 48
Sample Input 2	Sample Output 2
456789 2	2 NOT FOUND NO ENERGY GENERATED

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

In the ancient cyber realm of **Aetherion Grid**, engineers have discovered a powerful **P-digit Signal Key (Number)** that controls the planetary communication network. This code must be analyzed to unlock its hidden energy. The system responds only to a special digit **N** embedded within the number.

As you examine the Signal Key from Right to Left:

- If the digit **N** is not found anywhere in the number, the system shuts down and no energy is generated.
- If the digit **N** is found anywhere in the number, the system activates. To compute the energy, the system uses a layered frequency analysis method. For each digit from 0 to 9, the system re-scans the entire number to count its occurrences and print frequency. The energy contribution of each digit is calculated as **digit × (frequency²)**. All contributions are summed to get the final energy.

Write a Java program to print whether **N** was found and the resulting energy value, or indicate that no energy is generated.

Sample Input 1	Sample Output 1
122331 3	Digit 0 -> Frequency: 0, Contribution: 0 Digit 1 -> Frequency: 2, Contribution: 4 Digit 2 -> Frequency: 2, Contribution: 8 Digit 3 -> Frequency: 2, Contribution: 12 Digit 4 -> Frequency: 0, Contribution: 0 Digit 5 -> Frequency: 0, Contribution: 0 Digit 6 -> Frequency: 0, Contribution: 0 Digit 7 -> Frequency: 0, Contribution: 0 Digit 8 -> Frequency: 0, Contribution: 0 Digit 9 -> Frequency: 0, Contribution: 0 Total Energy: 24 <u>Explanation:</u> 3 is found! Frequency count: 1 → 2 times, 2 → 2 times, 3 → 2 times $1 \times 2^2 + 2 \times 2^2 + 3 \times 2^2$ = 24
Sample Input 2	Sample Output 2
456789 2	2 NOT FOUND NO ENERGY GENERATED

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;
public class SetA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int signalKey = sc.nextInt();
        int N = sc.nextInt();
        int temp = signalKey;
        boolean found = false;
        // Check digit N from right to left
        while (temp > 0) {
            if (temp % 10 == N) {
                found = true;
            }
            temp /= 10;
        }
        if (found == false) {
            System.out.println("Digit " + N + " not found. No energy generated.");
        }
        else {
            int totalEnergy = 1;
            // For digits 0 to 9
            for (int d = 0; d <= 9; d++) {
                int freq = 0;
                temp = signalKey;
                // count frequency of digit d
                while (temp > 0) {
                    if (temp % 10 == d) {
                        freq++;
                    }
                    temp /= 10;
                }
                int contribution = d * freq;
                System.out.println("Digit " + d +
                    " -> Frequency: " + freq +
                    ", Contribution: " + contribution);
                // multiply only non-zero contributions
                if (contribution != 0) {
                    totalEnergy *= contribution;
                }
            }
        }
    }
}
```

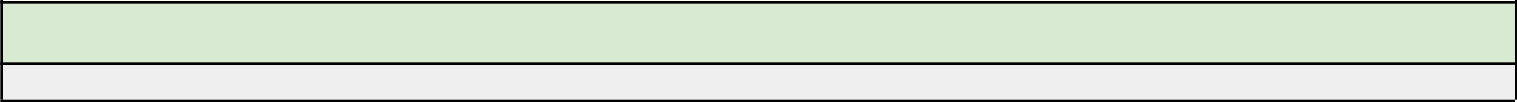
```
System.out.println("Total Energy: " + totalEnergy);  
}}}
```

SET - [B]

SOLUTION

```
import java.util.Scanner;  
public class SETB {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int signalKey = sc.nextInt();  
        int N = sc.nextInt();  
        int temp = signalKey;  
        boolean found = false;  
        //check if N exists  
        while (temp > 0) {  
            int digit = temp % 10;  
            if (digit == N) {  
                found = true;  
            }  
            temp /= 10;  
        }  
        if (!found) {  
            System.out.println("Digit " + N + " not found. No energy generated.");  
        }  
        else{  
            int totalEnergy = 0;  
            // process digits 0 to 9  
            for (int d = 0; d <= 9; d++) {  
                int freq = 0;  
                temp = signalKey;  
                // count frequency of digit d  
                while (temp > 0) {  
                    if (temp % 10 == d) {  
                        freq++;  
                    }  
                    temp /= 10;  
                }  
                int contribution = d * (freq * freq);  
                totalEnergy += contribution;  
            }  
        }  
    }  
}
```

```
        System.out.println("Digit " + d +  
            " -> Frequency: " + freq +  
            ", Contribution: " + contribution);  
    }  
    System.out.println("Total Energy: " + totalEnergy);  
}  
}
```



SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

Write a Java program that continuously takes input from the user using a loop. In each iteration, the user will enter a string and an integer d , where $0 \leq d < n$ and n is the length of the string. The program should perform a **left rotation** on the given string by d positions.

In a left rotation:

- The first **d characters** of the string are moved to the end.
- The remaining characters are shifted to the left.

After performing the rotation, the program should display the updated string. The process should continue until the user enters "END" as the string, at which point the program should stop execution.

Sample Input:	Sample Output:
Enter string: ABCD Enter d: 1 Enter string: ABCD Enter d: 2 Enter string: HELLO Enter d: 3 Enter string: END	BCDA CDAB LOHEL The program stopped.

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

Write a Java program that continuously takes input from the user using a loop. In each iteration, the user will enter a string and an integer d , where $0 \leq d < n$ and n is the length of the string. The program should perform a **right rotation** on the given string by d positions.

A right rotation means:

- moving the last **d characters** of the string to the beginning
- shifting the remaining characters to the right.

After performing the rotation, the program should display the updated string. The process should continue until the user enters "END" as the string, at which point the program should stop execution.

Sample Input:	Sample Output:
Enter string: ABCD Enter d: 1 Enter string: ABCD Enter d: 2 Enter string: HELLO Enter d: 3 Enter string: END	DABC CDAB LLOHE The program stopped.

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.print("Enter string: ");
            String s = sc.nextLine();

            if (s.equals("END")) {
                break;
            }

            System.out.print("Enter d: ");
            int d = sc.nextInt();
            sc.nextLine();

            int n = s.length();
            d = d % n;

            String temp = "";
            int i = 0;
            while (i < d) {
                temp += s.charAt(i);
                i++;
            }

            String result = "";
            i = d;
            while (i < n) {
                result += s.charAt(i);
                i++;
            }

            i = 0;
            while (i < temp.length()) {
                result += temp.charAt(i);
                i++;
            }

            System.out.println(result);
        }

        sc.close();
        System.out.println("The program stopped.");
    }
}
```

SET - [B]

SOLUTION

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.print("Enter string: ");
            String s = sc.nextLine();

            if (s.equals("END")) {
                break;
            }

            System.out.print("Enter d: ");
            int d = sc.nextInt();
            sc.nextLine();

            int n = s.length();
            d = d % n;
            String temp = "";
            int i = n-d;
            while (i < n) {
                temp += s.charAt(i);
                i++;
            }

            String result = "";
            i = 0;
            while (i < n-d) {
                result += s.charAt(i);
                i++;
            }

            i = 0;
            while (i < temp.length()) {
                result += temp.charAt(i);
                i++;
            }

            System.out.println(result);
        }

        sc.close();
        System.out.println("The program stopped.");
    }
}
```

}

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

The streaming service is analyzing viewing habits for the TV show **Friends**. A user is considered a **true Friends binge watcher** only if they watched episodes for at least **3 consecutive days** where the number of episodes watched each day was **strictly increasing**. Among all such increasing streaks, consider only the **longest streak**, and if two streaks have the same length, choose the one with the higher total episodes watched. Each element of a given integer array **watch** represents how many episodes of *Friends* a user watched each day. Write a Java program that prints "True Binge Watcher" if the selected streak has a total of **10 or more episodes**; otherwise print "Casual Watcher".

Sample Input 1: Given Array: <pre>int [] watch = {2, 3, 5, 6, 1, 2};</pre>	Sample Output 1: True Binge Watcher Explanation: Increasing streaks: 2, 3, 5, 6 → length = 4, sum = 16 1, 2 → length = 2 (ignored) Selected streak = 2, 3, 5, 6 sum = 16 >= 10
Sample Input 2: Given Array: <pre>int [] watch = {1, 2, 3, 0, 1};</pre>	Sample Output 2: Casual Watcher Explanation: Increasing streaks: 1, 2, 3 → length = 3, sum = 6 0, 1 → length = 2 (ignored) Selected streak = 1, 2, 3 sum = 6 < 10

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

The streaming service is analyzing viewing habits for the TV show **The Big Bang Theory**. A user is considered a true *The Big Bang Theory* binge watcher only if they watched episodes for **at least 3 consecutive days** where the number of episodes watched each day was strictly decreasing. Among all such **decreasing streaks**, consider only the **longest streak**, and if two streaks have the same length, choose the one with the higher total episodes watched. Each element of a given integer array **watch** represents how many episodes of *The Big Bang Theory* a user watched each day. Write a Java program that prints **"True Binge Watcher"** if the selected streak has a total of 12 or more episodes; otherwise print **"Casual Watcher"**.

Sample Input 1: Given Array: <pre>int [] watch = {8, 6, 4, 2, 5, 3, 1};</pre>	Sample Output 1: True Binge Watcher Explanation: Decreasing streaks are: 8, 6, 4, 2 → length = 4, sum = 20 5, 3, 1 → length = 3, sum = 9 Longest streak = 8, 6, 4, 2 Sum = 20 >= 12
Sample Input 2: Given Array: <pre>int [] watch = {5, 4, 2, 6, 2, 1};</pre>	Sample Output 2: Casual Watcher Explanation: Decreasing streaks are: 5, 4, 2 → length = 3, sum = 11 6, 2, 1 → length = 3, sum = 9 Both streaks have same length, so choose the one with the higher total episodes watched: sum = 11 <= 12

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
public class BingeWatcher {
    public static void main(String[] args) {
        int[] watch = {2, 3, 5, 6, 1, 2};

        int maxLength = 0;
        int maxSum = 0;

        int currentLength = 1;
        int currentSum = watch[0];

        for (int i = 1; i < watch.length; i++) {

            if (watch[i] > watch[i - 1]) {
                currentLength++;
                currentSum += watch[i];
            } else {
                if (currentLength >= 3) {
                    if (currentLength > maxLength ||
                        (currentLength == maxLength && currentSum > maxSum)) {
                        maxLength = currentLength;
                        maxSum = currentSum;
                    }
                }

                currentLength = 1;
                currentSum = watch[i];
            }
        }

        // checking last streak
        if (currentLength >= 3) {
            if (currentLength > maxLength ||
                (currentLength == maxLength && currentSum > maxSum)) {
                maxLength = currentLength;
                maxSum = currentSum;
            }
        }

        if (maxSum >= 10) {
            System.out.println("True Binge Watcher");
        } else {
            System.out.println("Casual Watcher");
        }
    }
}
```

SET - [B]

SOLUTION

```
public class BingeWatcher {
    public static void main(String[] args) {
        int[] watch = {8, 6, 4, 2, 5, 3, 1};

        int maxLength = 0;
        int maxSum = 0;

        int currentLength = 1;
        int currentSum = watch[0];

        for (int i = 1; i < watch.length; i++) {
            if (watch[i] < watch[i - 1]) {
                currentLength++;
                currentSum += watch[i];
            } else {
                if (currentLength >= 3) {
                    if (currentLength > maxLength ||
                        (currentLength == maxLength && currentSum > maxSum)) {
                        maxLength = currentLength;
                        maxSum = currentSum;
                    }

                    currentLength = 1;
                    currentSum = watch[i];
                }
            }
        }
        // Check the last streak
        if (currentLength >= 3) {
            if (currentLength > maxLength ||
                (currentLength == maxLength && currentSum > maxSum)) {
                maxLength = currentLength;
                maxSum = currentSum;
            }
        }

        if (maxSum >= 12) {
            System.out.println("True Binge Watcher");
        } else {
            System.out.println("Casual Watcher");
        }
    }
}
```